Retail Data Room Copilot

Revised product, UI and architecture document (v2)

Prepared for Prakhar Gupta, 8 November 2026. Revised after a critical review to reduce scope and timeline risk. Targets four Bain Capability Network (BCN) roles: Capabilities Tech Hub (Commercial Excellence, Agentic AI), Retail CoE (Associate, AI Data Engineer), PEG Alpha (Associate, AI Data Engineer) and PEG (Associate, AI Solutions Engineer).

1. Summary

An analyst creates a Data Room, uploads a retail sales spreadsheet and optional documents, and asks questions in one chat. A single LangGraph agent answers numeric questions with read-only SQL and text questions with pgvector search, showing its steps live. A checker verifies every number before the answer is shown. The UI is a single 3-pane dashboard, not a multi-page app. Column mapping uses RapidFuzz heuristics with an LLM fallback only when confidence is low. Ingestion runs in-process via FastAPI BackgroundTasks, not a separate queue.

Item	Decision
Domain	Retail: pricing and promotion, category and product profitability, customer analytics; also supports PE diligence and survey data
Inputs	CSV or Excel sales data; optional PDFs
Front end	Next.js, TypeScript, Tailwind, shadcn/ui, Recharts; one dashboard, no multi-page routing
Back end	FastAPI, LangGraph, Postgres with pgvector
Mapping	RapidFuzz heuristic first; LLM only below 70% confidence
Ingestion	FastAPI BackgroundTasks in-process; queue only if needed later
RAG	pgvector semantic search + metadata filter; no hybrid/reranker in v1
Agent	Single LangGraph loop with tools; no critic or multi-agent in v1
Quality layer	Upload validation, mapping confirmation, number checker, golden-set eval, run logging
Delivery	Docker Compose, GitHub Actions CI, Azure deployment
Estimated effort	3 to 4 weeks part-time (author estimate)
Biggest risk	Scope creep into 'chat with any file'

2. Coverage of the four job descriptions

Every requirement below is taken from the JD text you provided. 'Core' means part of the base build; 'Add-on' means an optional module.

Requirement	Capabilities Tech Hub	Retail CoE	PEG Alpha	PEG AI Solutions	Where it lives
Agent architecture (tools, memory, guardrails)	Yes	Yes	Yes	Yes	LangGraph agent (core)
RAG and vector search	Yes	Yes	Yes	Nice to have	pgvector (core)
Python, SQL, APIs	Yes	Yes	Yes	Yes	FastAPI + Postgres (core)
Evaluation, monitoring, guardrails	Yes	Yes	Yes	Yes	Golden set, checker, Eval panel (core)
ETL / data quality pipelines	Yes	Yes	Yes	-	Ingestion (core)

Requirement	Capabilities Tech Hub	Retail CoE	PEG Alpha	PEG AI Solutions	Where it lives
Docker, CI/CD, cloud	Productionise	Yes	Yes	Nice to have	Compose, Actions, Azure (core)
Front end React/Next	-	Nice to have	-	Nice to have	Next.js UI (core)
Neo4j knowledge graph	-	Yes	-	-	Add-on
Azure services (Blob, Redis)	-	Yes	Cloud	Cloud	Add-on
PySpark, Snowflake, Airbyte	-	PySpark/SQL	Familiar with	-	Add-on (light)
Survey validation	-	-	Context	-	Add-on
OpenAI / Anthropic APIs, prompt framework	GenAI	LLMs	LLM APIs	Core	Provider switch module
Tableau-style insight delivery	Yes	-	-	-	CSV export, charts

Note: the JDs set experience ranges (2 to 5 years for Tech Hub, 2 to 4 for Retail and PEG Alpha; PEG AI Solutions asks 2 to 4 years with an undergraduate degree or 0 to 2 with a postgraduate degree). A project supports your case but does not replace professional experience, so present it as evidence of skills. All four roles are listed in New Delhi.

3. Users, goals and scope

Primary user

A consulting analyst or associate who receives client or internal data as Excel/CSV and needs fast, defensible answers.

Goals

- Answer commercial questions from uploaded data with traceable numbers.
- Reject or repair bad files safely and explain why.
- Show the agent's work in the UI so users can trust or challenge it.
- Measure quality with a golden set and show it in the right-hand panel.
- Run locally with Docker and deploy cheaply to Azure.

Non-goals for v1

- Arbitrary spreadsheets (payroll, wide pivots, free-form reports).
- Multi-tenant enterprise auth, SSO, role hierarchies.
- Write access to data, automated pricing decisions, forecasting.
- Web search in the default answer path.
- Hybrid BM25 + cross-encoder reranking.

Example questions

- Which sub-categories lose money, and is it linked to discounts?
- Top 10 customers by profit and their share of total profit.
- Which promotions lost money last quarter?
- Summarise what the uploaded category report says about private label.

4. UI: one dashboard, three panes

Pane	Contents	Backend work
LEFT (Data & Health)	Drag-and-drop upload, per-file progress card (safety, structure, profiling, mapping), mapping confirmation modal, data health summary, quarantine list, download quarantine CSV	Store raw file, run BackgroundTask, stream status; mapping saved on confirm
CENTER (Chat)	Message box, suggested questions, streaming answer, inline badges (verified, caveat), clickable source chips (SQL, doc passage, row count), thumbs up/down, CSV export	Agent run over SSE, checker result, citations, feedback log, export endpoint
RIGHT (Trace & Eval, collapsible)	Live LangGraph steps (schema, KB read, SQL, doc search, checker), execution time, number verification result; tab with golden-set score card, latency, cost, failure types	Emit step events; read run logs and eval results

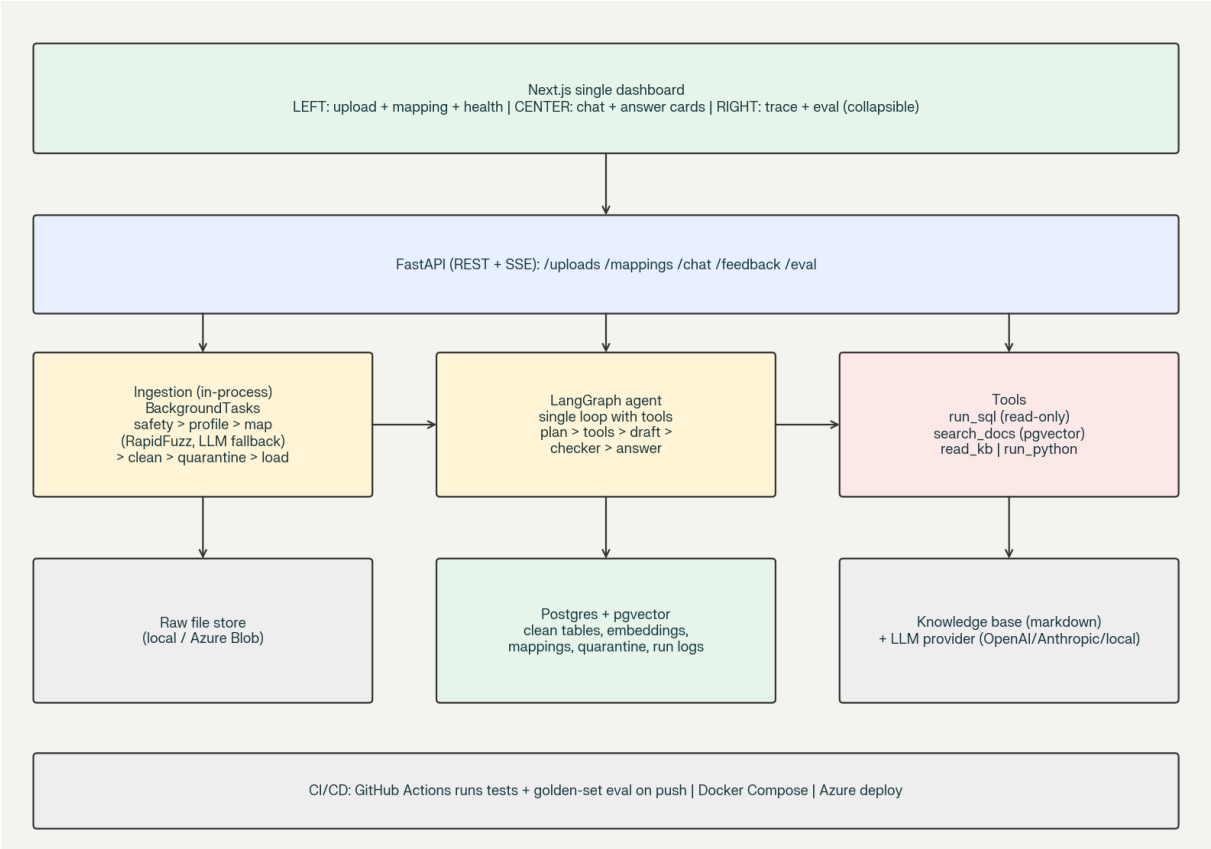
UI principles

- Show uncertainty: confidence badges, 'could not verify' warnings, clear refusals.
- Every number clickable to its SQL and row count.
- Progressive disclosure: simple answer first, trace and sources one click away.
- Empty, loading and error states designed for every pane.
- Keyboard-friendly, responsive, accessible colours and labels.

Why one dashboard

Nine screens were cut to three panes because frontend edge cases (SSE re-renders, mapping dropdowns, table pagination) can consume most of the build time. A single dashboard keeps the product cohesive and keeps the focus on the agent and data layer.

5. System architecture



Design decisions

Question	Decision	Reason
Source of truth for numbers	Clean Postgres tables, never the raw file at question time	Two copies can disagree; raw kept for debugging and lineage
Vector store	pgvector in the same Postgres	One database, lower cost and ops; swap to Azure AI Search later if needed
Knowledge base	Markdown files first, graph later	Easy to debug and version; Neo4j only as an add-on
Ingestion orchestration	FastAPI BackgroundTasks, in-process	Simple, no queue infra; add a queue only if load demands it
Agent shape	Single LangGraph loop with tools	Fast, cheap, easy to debug; split only if evals show a need
Column mapping	RapidFuzz heuristic first, LLM fallback below 70% confidence	Fast, cheap, deterministic; LLM only for ambiguous cases
RAG	pgvector semantic search + metadata filter	Sufficient for v1; hybrid/rerank later if retrieval metrics demand it
Web search	Off by default, labelled if enabled	Keeps answers grounded
Streaming	Server-sent events	Simple, one-way, works well for step and token streams

6. Data model

Table	Key columns	Purpose
workspaces	id, name, created_at	Data Room container

Table	Key columns	Purpose
uploads	id, workspace_id, original_name, stored_name, sha256, size, status, created_at	Raw file registry and status
column_mapping	upload_id, source_column, canonical_column, confidence, method (fuzzy/llm), confirmed_by_user	Lineage of mapping decisions
sales_clean	workspace_id, upload_id, order_date, customer_id, product_id, quantity, revenue, unit_price, discount_pct, cost, profit, segment, region, category, sub_category, channel	Canonical table the agent queries
quarantine	upload_id, row_number, reason, raw_json	Rejected rows with reasons
documents / chunks	doc_id, chunk_id, text, embedding, page, section, metadata	RAG content with metadata filters
runs	id, question, plan, sql_list, answer, checker_result, latency_ms, tokens, cost, model, created_at	Observability and eval source
feedback	run_id, rating, reason	Human-in-the-loop signal
eval_results	eval_run_id, question_id, passed, metrics_json	Eval panel data

Canonical required concepts: a date, a customer or product identifier, and a revenue or quantity measure. Cost or profit is optional, and margin questions are refused without it.

7. Ingestion and rejection

Layer	Checks	Outcome
1 File safety	Extension allow-list (csv, xlsx); content sniffing rather than trusting declared type; size cap; generated stored filename; reject macro-enabled files; parse in a sandboxed worker	Reject
2 Structure	Non-empty, more than one column, row/column caps, header-row detection, multi-sheet handling	Reject or ask
3 Domain fit	After mapping, requires date-like, revenue or quantity, and identifier at adequate confidence	Reject with what is needed
4 Row validation	Date parse, numeric types, negatives, duplicates, impossible discounts	Quarantine with report
5 Text safety	All cells treated as data; prefix cells starting with = + - @ on any export; data passed to the model as labelled data, never instructions	Sanitise

These layers follow standard guidance: OWASP file-upload practices (allow-listing, size limits, renamed files, not trusting declared content type) and CSV/formula-injection mitigation.

Column mapping: heuristic first

```
1. Normalise header (lowercase, strip, collapse spaces)
2. RapidFuzz ratio against canonical list (order_date, sales, profit,
   discount_pct, customer_id, product_id, quantity, unit_price, ...)
3. If best score >= 70%: auto-map, show badge
4. If below 70%: LLM proposes mapping using column_hints.md + samples
5. User confirms or edits; mapping saved with method recorded
```

RapidFuzz provides fast string matching with Levenshtein and token-based metrics, and supports a score cutoff for filtering [web:49][web:60]. This avoids LLM cost and latency for the common case.

Six deliberate test files

File	Expected behaviour
1 Clean sales table	Accepted, auto-mapped
2 Junk rows above header	Header detected or user asked
3 Messy names and text numbers	Mapped with confirmation, cleaned
4 Empty or single-column file	Rejected with reason
5 Unrelated data (payroll)	Rejected at domain-fit layer
6 Formula-like and prompt-injection text in cells	Stored as inert data; agent unaffected

8. Agent design

Tools

Tool	Behaviour	Guardrail
get_schema	Columns, types, mapping, samples	-
run_sql	SELECT only against the workspace's clean data	Read-only DB role, parser check, row limit, timeout, workspace filter
search_docs	pgvector semantic search with metadata filter, returns passages with page numbers	Workspace-scoped
read_kb	Opens a knowledge-base file by path	Restricted to kb/
run_python	pandas/sklearn for segmentation, correlation, cohorts	Sandbox, no network, memory/time limits
web_search (optional)	External context	Off by default; labelled external

Loop

```
1. Read question; get_schema; read_kb per routing rules
2. Plan; check required columns exist (else refuse with reason)
3. run_sql and/or search_docs; inspect results
4. Optional run_python for stats or segmentation
5. Draft answer using only numbers present in tool outputs
6. Checker compares every number and claim with tool outputs
7. Emit answer + SQL + row counts + sources + caveats or refusal
```

Knowledge base layout

```
kb/
  README.md          # routing: which file for which need
  metrics.md         # margin, discount rate, AOV, churn, RFM
  column_hints.md    # header variants -> canonical columns
  playbooks/discount_leakage.md, segmentation_rfm.md, product_profitability.md
  caveats.md         # when not to answer
```

Guardrails

- No write or DDL; read-only database role.
- Refuse questions the data cannot support.
- State correlation, not causation, unless supported.
- Do not infer sensitive personal attributes; mask identifiers if data looks personal.
- Maximum steps and token budget per question.

- Cell text and document text never treated as instructions.

Prompt and model layer

Prompts live in versioned files with typed structured outputs. A provider interface allows swapping OpenAI, Anthropic or a local model (Ollama/vLLM) via configuration, and the eval harness can compare them on quality, latency and cost. This is the PEG AI Solutions add-on.

9. API and streaming contract

Endpoint	Method	Purpose
/workspaces	POST, GET	Create and list Data Rooms
/workspaces/{id}/uploads	POST	Upload a file; returns upload id
/uploads/{id}/events	GET (SSE)	Stream ingestion status
/uploads/{id}/mapping	GET, PUT	Read proposed mapping; save user edits and confirm
/uploads/{id}/health	GET	Health report and quarantine download
/workspaces/{id}/chat	POST (SSE)	Ask a question; stream steps and answer
/runs/{id}/feedback	POST	Thumbs and reason
/eval/runs	GET, POST	List and trigger evaluation runs
/results/{id}/export	GET	CSV export

Chat stream event types: **step** (tool name, status), **sql** (query, row count), **source** (document, page, snippet), **token** (partial answer), **check** (verified or caveats), **done** (run id, cost, latency), **error**.

10. Evaluation and monitoring

Metric	How measured	Suggested target
SQL correctness	Result vs hand-verified answer on golden set	85% or higher
Numeric faithfulness	Share of answer numbers traceable to tool output	98% or higher
Retrieval hit rate	Expected passage in top-k for document questions	85% or higher
Refusal accuracy	Correct refusal of unanswerable questions and bad files	90% or higher
Mapping accuracy	Correct canonical mapping across test files and datasets	90% or higher
Latency and cost	Logged p50/p95, tokens, cost per question	Report; compare models
Safety	Prompt-injection cells, write-query attempts	0 successful

- Golden set of 25 to 50 questions across Superstore, Online Retail II and your documents, with verified answers.
- Regression run in CI on every push; fail the build if scores drop beyond a threshold.
- LLM-as-judge only for narrative quality, never for numbers.
- Weekly drift check: rerun the golden set and chart scores in the right pane.
- Record before/after results for the checker and for model swaps: these are your interview numbers.

11. Security and privacy

- Per-workspace data isolation in queries and retrieval.
- Secrets in environment variables or a secrets manager, never in the repo.
- Uploads limited by size and type, stored under generated names, scanned and parsed in a sandbox.
- Rate limits and request size limits on the API.
- Audit log of questions, SQL run and files uploaded.
- Only public or synthetic data in the published demo.

12. Technology stack

Layer	Core	Optional / later
UI	Next.js, TypeScript, Tailwind, shadcn/ui, Recharts, SSE	Auth, TanStack Table
API	FastAPI, Pydantic, BackgroundTasks	Redis cache, Celery queue
Agents	LangGraph single loop	Azure AI Agent Service comparison
Database	Postgres + pgvector	Snowflake adapter, Azure AI Search
Graph	-	Neo4j product hierarchy
Data processing	pandas, SQL	PySpark job for large file
Ingestion jobs	BackgroundTasks	Airbyte or Airflow demo
Models	One hosted provider + one local	OpenAI/Anthropic switch, reranker
Observability	runs table, Langfuse optional	Dashboards
Packaging	Docker Compose, pytest, GitHub Actions	Azure Container Apps / App Service, Blob storage

13. Repository structure

```
retail-data-room-copilot/
  apps/web/            # Next.js single-dashboard UI
  apps/api/            # FastAPI app, routes, SSE, BackgroundTasks
  packages/ingest/     # safety, profiling, RapidFuzz mapping, cleaning, quarantine
  packages/agent/      # LangGraph graph, tools, checker, prompts
  packages/rag/        # chunking, embeddings, pgvector search
  kb/                  # knowledge base markdown
  eval/                # golden set, runner, reports
  data/samples/        # six test files + public dataset loaders
  infra/               # docker-compose, Azure config, CI
  docs/                # architecture, ADRs, demo script
```

14. Datasets and sample content

Source	Use	Limitation
Superstore Sales (Kaggle)	Discount and profit questions; primary dataset	About 10k rows; check licence of the copy used
Online Retail II (UCI)	Excel upload demo, segmentation (UK online retailer, Dec 2009 to Dec 2011)	No cost or profit
Olist Brazilian E-commerce (Kaggle)	Multi-table joins and larger ETL (about 100k orders, 2016 to 2018)	No discount or profit column

Source	Use	Limitation
Self-written retail PDFs	RAG demo: category notes, mock diligence memo	Synthetic
Synthetic survey file	Survey validation add-on with seeded bad responses	Synthetic

None is real client data; describe them as stand-ins.

15. Add-on modules: pick one or two

Do not build all six. Pick based on the first role you apply to.

Add-on	Best target role	Effort	Tip
Survey validation	PEG Alpha	Low (1-2 days)	Detect speeders, straight-liners, duplicates; aligns with PEG Alpha's Survey CoE focus
Provider switch and model comparison	PEG AI Solutions	Low (1 day)	LiteLLM or clean interface; run golden set across two models, log cost/latency in Eval panel
Tableau/structured CSV export	Capabilities Tech Hub	Very low (0.5 day)	Endpoint converting SQL output to formatted CSV/Excel
Neo4j knowledge graph	Retail CoE	Medium/high (3-5 days)	Category > sub-category > product > brand graph; cut if time is short
PySpark / large-file transform	PEG Alpha	Medium (2-3 days)	One standalone job on Olist; show script and benchmark logs, no live cluster

16. Revised 4-week build roadmap

Week	Deliverable
1	Postgres + Superstore; read-only run_sql tool; 25-question golden set; number checker; CI eval script
2	Ingestion: parsing, header detection, RapidFuzz mapping with LLM fallback, quarantine; six test files pass
3	Single-dashboard Next.js UI; SSE streaming; trace panel; data health; answer cards with source chips
4	One role-specific add-on; Docker Compose; GitHub Actions CI running eval on push; 2-minute demo video

If time runs short

Cut in this order: Neo4j, PySpark, Redis, PDF support. Keep the SQL path, checker, Eval panel and upload validation, because they carry the most signal across all four roles.

17. Risks and mitigations

Risk	Mitigation
Scope creep	Declare supported data shape; reject the rest

Risk	Mitigation
Wrong column mapping gives silent errors	Confirmation step; mapping shown with answers; method recorded
LLM invents numbers	Checker plus faithfulness metric in CI
Unsafe SQL or code	Read-only role, parser checks, sandbox, timeouts
Prompt injection via cells or documents	Treat content as data; dedicated test file; no tool calls driven by content
Cost overrun	Token budgets, cached schema, RapidFuzz for mapping, local model option
UI takes longer than expected	One dashboard, not nine screens; reuse shadcn/ui components
Looks like a generic chat-with-file demo	Lead with the trace panel, rejection tests and Eval panel numbers

18. Demo script and interview talking points

Two-minute demo

- Upload a clean file and a bad file; show the rejection and the quarantine report.
- Confirm a column mapping (RapidFuzz badge, or LLM fallback).
- Ask which sub-categories lose money; show the live trace, chart, SQL chip and verified badge.
- Ask a question the data cannot answer; show the refusal.
- Ask a document question; show the source passage.
- Open the right pane and show scores before and after the checker.

Talking points

- Why SQL is the source of truth and the raw file is not re-read.
- Where the agent failed and how the checker changed the numbers.
- Local versus hosted model: quality, latency and cost per question.
- How you would extend it to a CoE's recurring workflow, and what would change at scale (auth, tenancy, observability).
- How you used AI coding tools and kept quality with tests and review.

19. Decision checklist

- You can commit 3 to 4 part-time weeks before applying.
- You are comfortable owning evaluation and testing, not only the demo.
- You will keep v1 to one table shape plus optional documents.
- You can explain every design choice, including failures.

If time is short, ship weeks 1 to 3 plus a minimal UI. That still demonstrates grounding, guardrails, validation and evaluation.

Open questions

- Which hosted model and monthly budget cap?

- Will the repo and demo video be public?
- Which role are you applying to first, and therefore which add-on?
- Do you want authentication in v1, or a single-user local mode?

Notes: effort estimates, targets and stack choices are the author's suggestions, not requirements from Bain. Job requirements are taken from the job descriptions you supplied. Security guidance reflects OWASP file-upload and CSV-injection recommendations; dataset facts reflect public dataset descriptions; mapping approach uses RapidFuzz, an open-source fuzzy string matching library.